

REPORTE DE ANALISIS TECNICO - RENRI

Fecha del analisis: 2026-03-30

Alcance: revision de la base actual en 'C:\Users\LENOVO\Documents\RENRI'

Estado del arbol: hay cambios locales sin commit; el reporte describe el estado actual del workspace, no necesariamente el ultimo commit limpio.

RESUMEN EJECUTIVO

RENRI tiene una base de producto ambiciosa y una estructura bastante buena para crecer: multi-tenant, agenda, pagos, portal cliente, portal publico, inventario, pedidos, turnos y Stripe Connect. La direccion del producto esta clara.

El problema es que el proyecto todavia no esta en un estado confiable para produccion. Hoy conviven buenas decisiones de arquitectura con errores graves de autorizacion, flujos publicos rotos, deuda de calidad y documentacion desactualizada. El resultado es un sistema con potencial alto, pero con riesgo operativo alto.

Diagnostico general:

- Base tecnica: buena
- Madurez de producto: media
- Madurez operativa: baja
- Riesgo de seguridad: alto
- Riesgo de regresion: alto

Validaciones ejecutadas:

- 'npm run lint': falla con 28 errores y 6 warnings
- 'npm run build': falla por error de tipos en 'src/app/portal/turno/[tenantSlug]/page.tsx:53'
- Busqueda de pruebas automatizadas: no se detectaron archivos 'test' o 'spec'
- CI/CD: no existe carpeta '.github/'

LO BUENO

- La separacion principal del proyecto es clara: 'src/app', 'src/actions', 'src/db', 'src/components', 'src/lib'. Eso facilita mantenimiento y onboarding.
- La capa de base de datos esta bien centralizada con Drizzle y un cliente lazy en 'src/db/index.ts:5-41'.
- El modelado multi-tenant existe de forma explicita en esquema y auth. Ejemplos: 'src/db/schema/tenants.ts:29-63', 'src/auth.ts:20-58', 'src/auth.config.ts:33-71'.
- Hay validacion con Zod centralizada para varias entidades en 'src/lib/schemas.ts:10-116'.
- Se ve intencion de seguridad en cabeceras HTTP globales dentro de 'next.config.mjs:14-24'.
- El flujo de reservacion publica de citas usa transaccion y lock asesoramiento para evitar doble reserva. Eso es una buena decision en 'src/actions/portal.ts:143-160'.
- La integracion de pagos no esta improvisada: existe soporte para Stripe y Stripe Connect con separacion utilitaria en 'src/lib/stripe.ts'.
- El sistema usa revalidacion despues de mutaciones y ya contempla portales distintos para negocio y cliente.
- El alcance funcional ya es amplio. No es un CRUD pequeno; el proyecto ya modela casos reales

de negocio.

LO MALO

HALLAZGOS CRITICOS

1. Stripe Connect puede vincularse sin validar sesion ni firmar correctamente el 'state'.

Evidencia:

- 'src/lib/stripe.ts:62-73' genera 'state' como Base64 simple.
- 'src/app/api/stripe/connect/callback/route.ts:30-38' confia en ese 'state'.
- 'src/actions/stripe-connect.ts:70-85' guarda la cuenta conectada sin verificar usuario

autenticado.

Impacto:

- Riesgo serio de asociar una cuenta de Stripe a un tenant ajeno.

Cambio sugerido:

- Reemplazar 'state' por token firmado con expiracion, verificar sesion del owner en callback y comparar tenant contra sesion antes de persistir.

2. Existe un endpoint de debug abierto que expone todos los tenants.

Evidencia:

- 'src/app/api/test/route.ts:5-7'

Impacto:

- Fuga innecesaria de datos de negocio y superficie de ataque.

Cambio sugerido:

- Eliminar el endpoint o protegerlo con rol 'SUPER_ADMIN' y deshabilitarlo fuera de desarrollo.

3. El flujo publico de turnos no es realmente publico y hoy rompe build.

Evidencia:

- UI publica llama 'createTurn' en 'src/app/portal/turno/[tenantSlug]/page.tsx:46-55'
- 'createTurn' exige sesion en 'src/actions/turns.ts:56-58'
- build falla en 'src/app/portal/turno/[tenantSlug]/page.tsx:53'
- el flujo nuevo repite la misma dependencia en 'src/app/portal/[tenantSlug]/turno/portal-turno-client.tsx:46-57'

Impacto:

- La fila publica no es confiable y parte del producto no compila.

Cambio sugerido:

- Separar 'publicCreateTurn' de 'createTurn', permitir identidad anonima o tokenizada, tipar 'Turn.number' de forma consistente y eliminar una de las dos rutas duplicadas.

4. El cron de recordatorios probablemente esta roto.

Evidencia:

- 'src/app/api/cron/reminders/route.ts:20' llama 'getUpcomingAppointments(24)'
- 'src/actions/portal.ts:331-333' exige usuario autenticado para esa consulta

Impacto:

- Los recordatorios automatizados no deberian funcionar desde cron sin sesion.

Cambio sugerido:

- Mover la consulta a una funcion interna de servidor sin 'requireAuth', y dejar la proteccion en el route handler via 'CRON_SECRET'.

5. El portal cliente puede reapropiar citas por email sin limitar por tenant.

Evidencia:

- 'src/actions/client-portal.ts:103-115'

Impacto:

- Riesgo de reasignacion global de citas entre tenants si coincide el correo.

Cambio sugerido:

- Restringir el auto-claim al negocio activo y registrar auditoria de la reasignacion.

6. Hay acciones sensibles de usuarios con autorizacion insuficiente.

Evidencia:

- 'src/actions/users.ts:84-96' crea clientes en cualquier 'tenantId' recibido

- 'src/actions/users.ts:108-125' verifica o elimina clientes con solo estar autenticado

Impacto:

- Un usuario autenticado podria operar datos de otros tenants.

Cambio sugerido:

- Exigir rol 'OWNER'/'ADMIN', verificar pertenencia al tenant y auditar operaciones destructivas.

HALLAZGOS ALTOS

1. El checkout de productos descuenta stock antes de confirmar el pago.

Evidencia:

- 'src/actions/checkout.ts:59-120'

Impacto:

- Carritos abandonados o pagos fallidos dejan inventario reducido.

Cambio sugerido:

- Reservar stock temporalmente o descontarlo solo al recibir 'payment_intent.succeeded'.

2. Las citas pendientes bloquean horarios aunque el pago nunca se complete.

Evidencia:

- 'src/actions/portal.ts:97-126' considera ocupada cualquier cita no cancelada

- 'src/actions/portal.ts:259-272' crea la cita en estado 'pending'

Impacto:

- Slots quedan bloqueados indefinidamente.

Cambio sugerido:

- Introducir expiracion para citas pendientes y job de limpieza o confirmacion temporal.

3. La pantalla de APIs promete cifrado de llaves, pero el codigo no lo hace y ademas esas llaves ni siquiera alimentan el runtime real.

Evidencia:

- 'src/app/dashboard/configuracion/apis/apis-form.tsx:47-50' afirma que las llaves se almacenan encriptadas

- 'src/actions/tenant.ts:102-106' guarda el JSON validado directamente en 'billingSettings'

- busqueda de uso: 'stripePublicKey', 'stripeWebhookSecret' y 'billingSettings' no alimentan 'src/lib/stripe.ts' ni 'src/app/api/webhooks/stripe/route.ts'

Impacto:

- Riesgo de seguridad, falsa promesa al usuario y feature a medio cablear.

Cambio sugerido:

- O bien eliminar esa UI hasta implementar KMS/secret manager real, o cifrar de verdad y hacer que el runtime use esos secretos.

4. La cancelacion publica de citas depende solo del 'appointmentId'.

Evidencia:

- 'src/app/portal/cancel/[appointmentId]/page.tsx:42-68'
- 'src/actions/client-portal.ts:196-248'

Impacto:

- Cualquier persona con el enlace puede cancelar; no hay expiracion ni firma.

Cambio sugerido:

- Usar token firmado, de un solo uso, con expiracion y revocacion.

5. Hay textos con codificacion rota en UI, metadata, logs y documentos.

Evidencia:

- 'src/app/layout.tsx:20-32'
- 'src/components/error-boundary.tsx:12-13', ':32', ':70-80'
- 'src/lib/logger.ts:2-4', ':52'
- 'IMPROVEMENTS.md:1-18'

Impacto:

- Mala percepcion de calidad, riesgo SEO y errores visibles al usuario final.

Cambio sugerido:

- Normalizar codificacion UTF-8 y revisar todos los archivos afectados.

6. El hook realtime recrea el cliente de Supabase en cada render.

Evidencia:

- 'src/hooks/use-turns-realtime.ts:16'
- dependencia del effect: 'src/hooks/use-turns-realtime.ts:36-69'

Impacto:

- Re-suscripciones innecesarias, mas consumo y comportamiento dificil de depurar.

Cambio sugerido:

- Crear el cliente una sola vez por componente con 'useState' o memo estable.

7. La busqueda de citas se aplica despues de paginar.

Evidencia:

- consulta paginada en 'src/actions/appointments.ts:69-81'
- filtrado por texto en memoria en 'src/actions/appointments.ts:85-94'

Impacto:

- Resultados incompletos y totales inconsistentes.

Cambio sugerido:

- Llevar la busqueda a SQL antes de 'limit/offset'.

8. 'stripeServer' usa un secreto dummy cuando falta la variable de entorno.

Evidencia:

- 'src/lib/stripe.ts:4-7'

Impacto:

- Misconfiguraciones silenciosas y errores mas dificiles de detectar.

Cambio sugerido:

- Fallar en arranque si 'STRIPE_SECRET_KEY' no existe.

9. El portal usa 'next/font/google', lo que agrega dependencia externa al build.

Evidencia:

- 'src/app/portal/[tenantSlug]/layout.tsx:1-12'
- 'src/app/portal/cancel/[appointmentId]/page.tsx:3-17'

Impacto:

- Builds menos portables y mas fragiles en entornos restringidos.

Cambio sugerido:

- Mover esas fuentes a local assets o asumir fallo controlado.

HALLAZGOS MEDIOS

1. El middleware ya usa una convencion deprecada.

Evidencia:

- warning de build sobre 'src/middleware.ts'

Cambio sugerido:

- Migrar a 'proxy' segun la guia actual de Next.

2. Hay desalineacion de versiones base.

Evidencia:

- 'package.json:40-45' usa 'next' 16 y 'react' 18
- 'package.json:64-66' usa 'eslint-config-next' 14

Impacto:

- Mantenimiento mas fragil y falsos positivos/negativos en tooling.

Cambio sugerido:

- Alinear 'next', 'eslint-config-next', 'react' y tipos al stack oficialmente soportado.

3. La documentacion principal no describe el producto real.

Evidencia:

- 'README.md:1-36' sigue siendo el template de 'create-next-app'

Impacto:

- Onboarding pobre y despliegues mas lentos.

Cambio sugerido:

- Reescribir README con arquitectura, variables de entorno, modulos, comandos, seeds y riesgos conocidos.

4. La documentacion interna esta desactualizada y contradice el estado actual.

Evidencia:

- 'IMPROVEMENTS.md:10-18' afirma que el proyecto compila
- 'IMPROVEMENTS.md:26-34' muestra una version de Stripe distinta a la actual

Impacto:

- Confusion tecnica y decisiones basadas en supuestos incorrectos.

Cambio sugerido:

- Depurar o archivar esa documentacion y dejar una sola fuente de verdad.

LO QUE FALTA IMPLEMENTAR

- Suite de pruebas automatizadas. No detecte pruebas unitarias, integracion ni e2e.
- Pipeline de CI/CD. No existe '.github/' con validaciones obligatorias.
- Tracking de errores real. Hay TODOs explicitos en 'src/components/error-boundary.tsx:19' y 'src/lib/logger.ts:52'.
- '.env.example' o guia completa de configuracion. Solo existe '.env.local'.
- Rate limiting y anti-abuso en flujos publicos. Inferencia: no encuentre implementacion al revisar booking, turnos publicos, checkout ni endpoints publicos.
- Expiracion y limpieza de estados 'pending' para citas, pagos y ordenes.
- Tokens firmados para acciones publicas sensibles como cancelacion.
- Integracion real de correos de confirmacion y cancelacion. Las funciones existen en 'src/lib/emails.ts:11-112', pero solo se usa el recordatorio del cron.
- Cierre real del rollout de RLS en Supabase. El propio documento 'supabase/IMPLEMENTATION-STATUS.md' lo marca como pendiente.

LO QUE NO ESTA BIEN IMPLEMENTADO

- Autorizacion dispersa y manual en muchas server actions. Eso ya produjo inconsistencias graves.
- Logica de negocio mezclada con atajos de producto. Ejemplo: placeholder emails en 'src/actions/users.ts:93' y turnos apoyados en 'clientId'/'staffId' del usuario actual en 'src/actions/turns.ts:85-100'.
- Duplicacion funcional en rutas de turnos publicos: 'src/app/portal/[tenantSlug]/turno/...' y 'src/app/portal/turno/[tenantSlug]/...'.
- Claims de seguridad no respaldados por implementacion real en la configuracion de APIs.
- Higiene de codigo insuficiente: 'any', imports sin uso, hooks con dependencias incompletas, 'img' en vez de 'next/image', comentarios de "mock" y placeholders.

RECOMENDACIONES PRIORITARIAS

PRIORIDAD 0 - BLOQUEANTES ANTES DE PRODUCCION

- Corregir Stripe Connect callback: 'state' firmado, sesion obligatoria y validacion tenant-owner.
- Eliminar o proteger '/api/test'.
- Arreglar el error de build de 'src/app/portal/turno/[tenantSlug]/page.tsx:53'.
- Rediseñar el flujo publico de turnos para que no dependa de 'requireAuth'.
- Corregir 'getUpcomingAppointments' para cron.
- Restringir 'users.ts' y el auto-claim del portal cliente por tenant y rol.

PRIORIDAD 1 - RIESGO OPERATIVO

- No descontar stock hasta pago confirmado o introducir reserva con expiracion.
- Expirar citas 'pending' que no se pagan.
- Reemplazar enlaces publicos de cancelacion por tokens firmados.
- Eliminar fallback 'sk_test_123' y fallar rapido por variables faltantes.
- Normalizar UTF-8 en todo el repo.

PRIORIDAD 2 - CALIDAD Y ESCALABILIDAD

- Agregar pruebas:
- Unitarias para helpers y autorizacion.
- Integracion para server actions criticas.
- E2E para booking, checkout, cancelacion y Stripe Connect.
- Crear CI con al menos 'lint', 'build' y pruebas.
- Unificar versionado de Next/React/ESLint.
- Reescribir README y consolidar documentacion tecnica.
- Implementar observabilidad real con Sentry o equivalente.

ORDEN RECOMENDADO DE IMPLEMENTACION

1. Seguridad y autorizacion.

2. Flujos de dinero e inventario.
3. Flujos publicos y cron.
4. Calidad automatizada: tests + CI.
5. Documentacion, encoding y limpieza general.

VEREDICTO FINAL

RENRI no es un proyecto malo; al contrario, tiene buena base, buen alcance y varias decisiones correctas. Lo que hoy le impide estar fuerte no es falta de vision, sino falta de cierre tecnico en seguridad, calidad operativa y consistencia de implementacion.

Si corriges primero autorizacion, pagos/stock, turnos publicos y automatizacion minima de calidad, el proyecto puede subir mucho de nivel en poco tiempo. Pero en su estado actual yo no lo consideraria listo para produccion sin antes cerrar los hallazgos criticos y altos de este reporte.